

Kerberos/NFS

역할: FARM/LAB에서 AD Kerberos로 사용자를 인증하고, 같은 UID/GID로 NFS 공유 스토리지에 접근하게 하는 구조와 운영 기준을 설명한다.

이 문서는 Kerberos/NFS 문서의 시작점이다. 처음 읽는 사람은 **설계**에서 전체 인증 흐름을 먼저 이해한 뒤 **운영**에서 생성·점검·복구 명령을 확인한다. 과거 장애의 증거와 재현 실험은 **디버깅 로그**에서 확인한다. 환경별 실제 값과 최종 실행 절차는 인프라 저장소의 FARM/LAB canonical runbook을 단일 기준으로 삼는다.

문서 구성

문서	읽어야 하는 경우	핵심 내용
현재 페이지	전체 범위와 문서 위치를 빠르게 확인할 때	목적, 원칙, 소유 경계
설계	인증이 어느 서버와 프로세스를 거치는지 이해할 때	keytab 발급, ticket 발급, RPCSEC_GSS, NAS 권한 판정, 설정 근거
운영	계정/컨테이너 생성, timer 확인, mount-KVNO 장애를 처리할 때	명령, 점검표, 모니터링 코드, 수동 repair/rotation
디버깅 로그	재현된 장애의 원인과 실험 조건·명령·증거를 확인할 때	장애별 증상, 가설, 재현, 판정, 복구·예방

한눈에 보는 구조

관리 서버(uidctl)

- AD DC: 사용자·RFC2307·keytab 생성
- 계산 호스트: root-only keytab + ccache refresh timer
- NAS/storage: NFS service keytab + 사용자 home
- container: keytab이 아닌 ccache만 사용

사용자 I/O

container process -> host kernel NFS client -> rpc.gssd

-> AD KDC ticket -> NAS svcgssd/NFS -> UID/GID 권한 판정

현재 구현의 중요한 경계는 다음과 같다.

- 사용자 keytab은 장기 자격증명이므로 계산 호스트의 `/etc/decs-krb/keytabs/<username>.keytab`에 `root:root0400`으로만 둔다.

- 컨테이너에는 keytab을 넣지 않는다. 호스트가 만든 `/run/user/<uid>/krb5cc` 와 읽기 전용 `/etc/krb5.conf` 만 bind mount한다.
- 컨테이너가 시작되기 전에 사용자별 `decs-krb-refresh@<username>.timer` 를 활성화하고 유효한 ccache를 한 번 발급한다.
- NFS mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 쓴다. FARM의 source는 `nas.farm.decs.internal:/volume1/share` 다.
- `addr=100.100.100.120` 은 FQDN이 해석된 실제 전송 주소로 사용할 수 있다. source 자체를 `100.100.100.120:/...` 로 바꾸면 안 된다.
- 기본 security flavor는 `sec=krb5` 다. `krb5i` 와 `krb5p` 는 무결성·암호화가 명시적으로 필요한 경로에서 성능 비용과 함께 선택한다.

Identity 불변 조건

한 사용자에게 대해 다음 숫자가 같아야 한다.

```
UID DB ubuntu_uid
= AD RFC2307 uidNumber
= NFS client에서 관측한 home owner UID
= container UID
```

primary GID도 같은 원칙 적용

이 조건을 확인하지 않고 NAS에서 `chown` 만 반복하면 AD, DB, NAS의 identity가 더 어긋날 수 있다. 계정 생성 흐름은 Docker와 DB를 확정하기 전에 AD/NAS/host identity와 실제 Kerberos NFS 쓰기를 검증한다.

모듈 소유 경계

작업	소유 모듈
AD 사용자·그룹, 사용자 keytab, host ccache timer, container 생성	<code>user-lifecycle</code>
공통 host Kerberos/NFS desired state와 fstab	<code>server-state</code>
NAS NFS service account·SPN·service keytab의 수동 변경	<code>kerberos-nfs</code>
GSS readiness, keytab drift 관측, mount 상태, canary, forensics	<code>monitoring</code>
ccache 소비, 시작 전 credential 확인, restricted sudo	<code>container-images</code>

관측 실패가 곧바로 AD password 변경이나 NAS service 재시작으로 이어지지 않게 읽기 전용 monitoring과 상태 변경 작업을 분리한다.

단일 기준 문서와 코드

- [FARM canonical runbook](#)
- [LAB canonical runbook](#)
- [user-lifecycle Kerberos 구현](#)
- [container 생성 구현](#)
- [host Kerberos/NFS desired state](#)
- [Kerberos/NFS keytab health check](#)
- [NFS GSS monitoring](#)

실제 endpoint, principal, mount option이 바뀌면 이 위키보다 canonical runbook과 코드 설정을 먼저 갱신하고, 검증 시각과 대상 host를 함께 기록한다.

Kerberos/NFS 설계

이 문서는 keytab이 만들어지는 시점부터 사용자가 NFS 파일을 읽고 쓰는 시점까지, 어느 서버의 어떤 객체와 프로세스가 관여하는지를 설명한다.

1. 설계 목표

- 비밀번호를 컨테이너에 저장하지 않고 사용자별 Kerberos 인증을 제공한다.
- DB, AD, 계산 호스트, NAS와 컨테이너가 같은 numeric UID/GID를 사용한다.
- NFS 서버는 FQDN 기반 service principal로 인증하고 client는 `sec=krb5` 로 RPCSEC_GSS context를 만든다.
- keytab 같은 장기 자격증명의 노출 범위를 host root로 제한한다.
- ticket 갱신과 컨테이너 lifecycle을 분리하여 컨테이너 재시작 중에도 credential을 안정적으로 유지한다.
- monitoring은 drift를 관측하되 공유 NAS와 AD를 자동으로 변경하지 않는다.

2. 전체 구성

다이어그램의 **굵은 바깥 제목은 실행 영역**이고, 영역 안 각 카드의 **굵은 첫 줄은 모듈 이름**이다. 카드 안의 **보관·참조 값 / 구성요소**는 명사형으로, **수행 동작**은 **주체와 동사가 있는 문장**으로 구분했다. 화살표 문장은 출발점의 모듈이 도착점의 모듈에 수행하는 동작을 뜻하며, 점선은 credential 참조나 읽기 전용 조회·관측을 뜻한다. AD/KDC는 모든 계산 host에 있는 것이 아니라 FARM/LAB 중 **AD DC 역할을 맡은 host에만** 있다. 다이어그램에는 핵심 값과 동작만 표시하고 상세 책임은 아래 표에서 설명한다.

flowchart TB

subgraph CONTROL_ZONE["외부 관리 · 관측"]

direction LR

M["관리 서버
—————
구성요소
uidctl · Ansible
수행 동작
계정 작업을 조정하고
설정과 keytab을 배포함"]

MON["Monitoring
—————
보관 값
metric · log · evidence
수행 동작
인증과 mount 상태를 검사하고
이상을 감지하면 경보함"]

end

subgraph CONTAINER_ZONE["사용자 컨테이너"]

USER_RUNTIME["사용자 Runtime 모듈
—————
참조 값
KRB5CCNAME
bind-mounted ccache
수행 동작
ticket으로 NFS I/O를 요청함"]

end

subgraph HOST_ZONE["FARM / LAB 호스트"]

direction LR

DIRECTORY_MODULE["AD / Kerberos Directory 모듈
—————
보관 값
user · group · SPN
RFC2307 UID/GID · KVN0
수행 동작
TGT와 service ticket을 발급함
<i>AD DC 역할 호스트에만 배치</i>"]

REFRESH_MODULE["사용자 Credential 갱신 모듈
—————
보관 값
user keytab (root:root 0400)
host ccache
수행 동작
timer가 ccache를 갱신하고
검증 후 원자 교체함"]

NFS_CLIENT_MODULE["NFS Client 모듈
—————
구성요소
kernel NFS client · rpc.gssd
수행 동작
호출 UID의 ccache로
GSS 인증을 적용해 RPC를 전송함"]

end

subgraph STORAGE_ZONE["NAS / Storage"]

direction LR

NFS_SERVER_MODULE["NFS Service 모듈
—————
보관 값 / 구성요소
NFS service keytab
svcgssd · NFS server
수행 동작
ticket과 권한을 검증해
NFS I/O를 처리함"]

STORAGE_ID_MODULE["Identity / 권한 모듈
—————
보관 값 / 구성요소
winbind · idmap · NFS export
user home · numeric UID/GID
수행 동작
principal을 UID/GID로 변환해
파일 권한을 판정함"]

end

M -->|"계정 · RFC2307을 만들고
keytab을 export함"| DIRECTORY_MODULE

M -->|"keytab을 root-only로 설치함"| REFRESH_MODULE

M -->|"home과 owner를 준비함"| STORAGE_ID_MODULE

REFRESH_MODULE -->|"keytab으로 TGT를 발급받음"| DIRECTORY_MODULE

USER_RUNTIME -.->|"host ccache를 참조함"| REFRESH_MODULE

USER_RUNTIME -->|"호출 UID로 I/O를 요청함"| NFS_CLIENT_MODULE

```

NFS_CLIENT_MODULE -->|"NFS service ticket을 발급받음"| DIRECTORY_MODULE
NFS_CLIENT_MODULE -->|"GSS 인증을 붙여<br/>NFS RPC를 전송함"| NFS_SERVER_MODULE
NFS_SERVER_MODULE -->|"principal의 권한을 조회함"| STORAGE_ID_MODULE
DIRECTORY_MODULE -. ->|"RFC2307 UID/GID를 반환함"| STORAGE_ID_MODULE

MON -. ->|"ticket · KVN0를 검사함"| DIRECTORY_MODULE
MON -. ->|"timer · ccache를 검사함"| REFRESH_MODULE
MON -. ->|"mount · rpc.gssd를 검사함"| NFS_CLIENT_MODULE
MON -. ->|"canary · keytab을 검사함"| NFS_SERVER_MODULE

classDef component fill:#ffffff,stroke:#64748b,stroke-width:1px,color:#0f172a
classDef external fill:#fff7ed,stroke:#d97706,stroke-width:2px,color:#431407
class M,MON external
class USER_RUNTIME,DIRECTORY_MODULE,REFRESH_MODULE,NFS_CLIENT_MODULE,NFS_SERVER_MODULE,STORAGE_ID_MODULE
component

style CONTAINER_ZONE fill:#eff6ff,stroke:#2563eb,stroke-width:3px
style HOST_ZONE fill:#f0fdf4,stroke:#16a34a,stroke-width:3px
style STORAGE_ZONE fill:#f5f3ff,stroke:#7c3aed,stroke-width:3px
style CONTROL_ZONE fill:#fff7ed,stroke:#d97706,stroke-width:2px
style USER_RUNTIME stroke:#60a5fa,stroke-width:2px
style DIRECTORY_MODULE stroke:#4ade80,stroke-width:2px
style REFRESH_MODULE stroke:#4ade80,stroke-width:2px
style NFS_CLIENT_MODULE stroke:#4ade80,stroke-width:2px
style NFS_SERVER_MODULE stroke:#a78bfa,stroke-width:2px
style STORAGE_ID_MODULE stroke:#a78bfa,stroke-width:2px

```

구성요소별 책임

영역	모듈	내부 구성요소	하는 일
외부 관리	계정 생성 orchestration	<code>uidctl</code> , Ansible runner	계정 생성 transaction을 조정하고 AD, host, storage에 필요한 상태를 준비
FARM/LAB 호스트	AD/Kerberos Directory	Samba AD DC, KDC, 사용자·그룹·SPN·RFC2307 객체	principal과 Unix identity 저장, keytab export, TGT/service ticket 발급; AD DC 역할 host에만 배치
FARM/LAB 호스트	사용자 Credential 갱신	user keytab, <code>decs-krb-refresh@.service/.timer</code> , host ccache	root-only keytab으로 ccache를 만들고 검증 후 원자 교체
FARM/LAB 호스트	NFS Client	kernel NFS client, <code>rpc.gssd</code>	호출 UID의 ccache로 RPCSEC_GSS context를 만들고 NFS RPC 전송

영역	모듈	내부 구성요소	하는 일
사용자 컨테이너	사용자 Runtime	사용자 process, bind-mounted ccache, KRB5CCNAME	keytab 없이 host가 갱신한 ticket을 사용하여 NFS 파일 I/O 수행
NAS/Storage	NFS Service	service keytab, svcgssd , NFS server	nfs/<fqdn>@<realm> service ticket을 수락하고 NFS 요청 처리
NAS/Storage	Identity/권한	Samba, winbind, idmap, NFS export와 filesystem	Kerberos principal을 RFC2307 UID/GID로 해석하여 최종 파일 권한 판정
외부 관측	Monitoring	readiness, KVNO checker, mount probe, canary, forensics	AD/host/storage 상태를 읽기 전용으로 수집하고 drift와 장애를 경보

3. 사용자 keytab 발급 흐름

keytab은 컨테이너 안에서 만들지 않는다. AD DC에서 export한 뒤 target 계산 호스트의 root-only 경로에 설치한다.

```
sequenceDiagram
    autonumber
    participant CLI as 관리 서버 uidctl
    participant AD as Samba AD DC
    participant Host as target 계산 호스트
    participant NAS as NAS/storage
    participant Docker as Docker container

    CLI->>AD: 사용자·그룹 생성/조회
    CLI->>AD: uidNumber, gidNumber, group membership 설정
    CLI->>AD: domain exportkeytab --principal=user@REALM
    AD-->>CLI: 임시 keytab
    CLI->>Host: /etc/decs-krb/keytabs/user.keytab<br/>root:root 0400 설치
    CLI->>NAS: home 생성 및 numeric UID:GID 준비
    CLI->>Host: refresh env/service/timer 설치·활성화
    Host->>AD: kinit -k -t user.keytab
    AD-->>Host: 사용자 TGT가 든 ccache
    Host->>Host: /run/user/uid/krb5cc로 원자 교체
    CLI->>Host: 실제 NFS 쓰기 검증
    CLI->>Docker: ccache dir와 krb5.conf만 bind mount
```

실제 생성 구현은 다음 순서로 동작한다.

1. DB/기존 AD/storage 값을 사용해 UID/GID를 선택한다.
2. AD user/group과 RFC2307 **uidNumber**, **gidNumber**를 보장한다.
3. AD DC에서 사용자 keytab을 임시 파일로 export하고 **0400**으로 만든다.

- target이 다른 host이면 Ansible `fetch` 와 `copy` 로 root-only keytab을 옮긴다.
- NAS/storage home을 동일 UID/GID로 준비한다.
- target host에 ccache directory, refresh helper, service/timer를 만든다.
- host에서 사용자 ccache를 발급하고 NFS owner와 실제 write/delete를 검증한다.
- 검증 후에만 컨테이너와 DB record를 확정한다.

관련 코드는 [AD identity와 keytab 생성](#), [container 생성 transaction](#), [credential 경로 모델](#)에서 확인한다.

4. 실제 NFS 인증 흐름

keytab 발급과 매 파일 접근의 인증은 서로 다른 흐름이다. 평상시 파일 I/O에서 컨테이너가 keytab을 직접 읽는 일은 없다.

```
sequenceDiagram
    autonumber
    participant P as container 사용자 process
    participant K as host kernel NFS client
    participant G as host rpc.gssd
    participant KDC as AD KDC
    participant S as NAS svcgssd/NFS
    participant ID as AD RFC2307 / winbind

    P->>K: open/read/write (호출 UID 포함)
    K->>G: RPCSEC_GSS credential upcall
    G->>G: /run/user/uid/krb5cc에서 사용자 TGT 확인
    G->>KDC: nfs/nas.farm.decs.internal service ticket 요청
    KDC-->>G: NFS service ticket
    G-->>K: GSS security context
    K->>S: NFSv4 RPC + Kerberos credential
    S->>S: service keytab으로 ticket 복호화
    S->>ID: principal을 RFC2307 UID/GID로 해석
    ID-->>S: numeric UID/GID와 group
    S-->>K: 파일 권한 판정 결과
    K-->>P: I/O 결과
```

인증 실패 위치에 따라 증상이 달라진다.

실패 위치	대표 증상
사용자 keytab/ccache	<code>kinit</code> 또는 <code>klist</code> 실패, 사용자 접근만 <code>Permission denied</code>
DNS/FQDN/SPN	<code>kvno nfs/<fqdn></code> 실패, IP source mount에서 principal 불일치
host machine keytab/ <code>rpc.gssd</code>	부팅 시 mount 실패, readiness의 keytab/kinit/rpc-gssd stage 실패

실패 위치	대표 증상
NAS service keytab/KVNO	여러 client가 동시에 GSS 실패, <code>kvno -k</code> 복호화 실패
RFC2307/ldap	인증은 되지만 owner가 <code>nobody</code> 이거나 UID/GID가 달라 쓰기 거부
NFS transport/kernel	<code>not responding</code> , D-state, mount probe timeout

5. Keytab과 ccache를 분리한 이유

구분	keytab	ccache
의미	principal의 장기 비밀키	이미 발급된 TGT/service ticket 묶음
새 ticket 발급	가능	제한된 renew 기간 안에서만 가능
유출 영향	rotation할 때까지 반복 악용 가능	ticket lifetime/renew lifetime에 제한
저장 위치	host root-only	<code>/run/user/<uid>/krb5cc</code> , 사용자 <code>0600</code>
container 전달	금지	bind mount하여 전달

컨테이너 삭제만으로 유출된 keytab을 무효화할 수 없기 때문에 image, Docker volume, Kubernetes Secret에 사용자 keytab을 그대로 넣지 않는다. 호스트 refresh service는 임시 ccache에서 `klist` 검증을 끝낸 뒤 소유권 `uid:gid`, mode `0600` 을 적용하고 최종 경로로 원자 교체한다.

현재 코드가 구현하는 대상은 Docker container다. Kubernetes Pod도 같은 원칙을 적용해야 하지만, Pod가 다른 node로 이동할 수 있으므로 단순 hostPath와 특정 node의 systemd timer에 의존해서는 안 된다. Pod 지원을 추가할 때는 node 배치, credential 발급 controller/CSI, rotation과 Pod 종료 시 정리를 별도 설계해야 한다.

6. Ticket 갱신 설계

`decs-krb-refresh@<username>.timer` 는 기본적으로 boot 2분 뒤 시작하고 사용자별 설정 주기(현재 일반적으로 1시간)마다 oneshot service를 실행한다.

유효 ccache 있음

- └─ renew deadline이 margin보다 멀 -> 임시 복사본에서 `kinit -R`
- └─ deadline 임박/renew 실패 -> root-only keytab으로 새 ticket 발급

발급 결과 -> `klist` 검증 -> `uid:gid 0600` -> 최종 ccache로 atomic move

기본 재발급 여유는 `DECS_KRB_REISSUE_BEFORE_SECONDS=86400` (24시간)이다. 기존 ticket의 PAC/group 정보는 renew 시 유지될 수 있으므로 AD group 변경 뒤에는 keytab을 이용한 fresh login을 한 번 강제해야 한다.

7. NFS service identity와 KVNO

FARM NFS SPN은 Synology domain member의 machine account `NAS$` 가 아니라 전용 AD service account `svc-nfs-farm` 이 소유한다.

```
nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL  
nfs/NAS@FARM.DECS.INTERNAL
```

과거에는 `NAS$` machine password가 주기적으로 변경될 때 KVNO도 바뀌어 NAS의 정적 NFS keytab과 어긋날 수 있었다. 이를 service identity와 domain membership의 lifecycle 결합 문제로 보고 NFS SPN을 전용 계정으로 분리했다.

현재 `svc-nfs-farm`에는 자동 password expiration/rotation timer가 없다. 따라서 “NAS KVNO가 지금도 주기적으로 바뀐다”가 아니라 **과거 자동 변경 문제를 전용 계정으로 제거했고, 현재 KVNO는 관리자가 명시적으로 rotation할 때만 바뀐다**가 정확한 운영 모델이다. 별도 5분 checker는 변경을 만들지 않고 AD KVNO와 두 NAS keytab의 일치만 확인한다.

NAS의 `svcgssd`는 FARM과 AILAB principal을 한 keytab에서 받아들이므로 `-p`로 FARM principal 하나만 강제하지 않는다. keytab repair 시에도 AILAB acceptor를 보존한다.

8. FQDN mount와 service principal

Kerberos의 NFS service identity는 host-based principal이다. FARM mount source는 다음처럼 principal의 hostname과 같아야 한다.

```
mount source: nas.farm.decs.internal:/volume1/share  
service SPN: nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL  
transport:   addr=100.100.100.120
```

`100.100.100.120:/volume1/share`를 source로 쓰면 client가 IP 기반 service identity를 찾으려 하거나 기대한 FQDN SPN과 연결하지 못할 수 있다. 반면 source를 FQDN으로 유지한 상태에서 runtime option에 `addr=100.100.100.120`이 보이는 것은 정상이다. 관리 SSH 주소 `192.168.2.30`은 NFS transport로 사용하지 않는다.

9. 보안 flavor와 권한 모델

- `sec=krb5`: 사용자/서비스 인증. RPC payload 무결성·암호화 wrapping 없음.
- `sec=krb5i`: 인증 + RPC payload 무결성.
- `sec=krb5p`: 인증 + 무결성 + privacy encryption.

현재 내부 FARM/LAB 기본은 `sec=krb5`다. packet capture에 payload가 포함될 수 있으므로 NFS forensics 산출물은 root-only로 보관한다.

Kerberos 인증 뒤 최종 파일 권한은 numeric UID/GID와 mode/ACL로 결정된다. 컨테이너 root가 host credential 경계를 우회하지 못하도록 Kerberos mode에서는 `DECS_USER_SUDO_MODE=restricted` 를 함께 사용한다. 다만 `restricted sudo`는 완전한 sandbox가 아니므로 강한 격리가 필요하다면 `sudo`와 `SYS_ADMIN` , host bind mount 범위도 별도로 줄여야 한다.

10. 설정과 구현 위치

관심사	기준 코드/문서
FARM endpoint, SPN, mount option, NAS keytab	FARM runbook
LAB topology와 rollout gate	LAB runbook
사용자 AD/keytab/ccache 명령 생성	commands.py
keytab·ccache·home 경로	paths.py
create-container transaction과 Docker bind	create_container.py
container credential 대기과 restricted sudo	entrypoint.sh
host machine keytab/readiness/fstab	kerberos_nfs_client_recovery.yml
NFS GSS readiness와 recovery	cluster-monitor-exporter scripts
NAS service keytab drift checker	check-nfs-keytab.sh

Kerberos/NFS 운영

이 문서는 현재 구현된 Docker/FARM·LAB 흐름의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

- 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
- 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
- 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
- mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.
- `findmnt` , `klist` , `kvno` , readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
- UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성 (추후 자동화 시스템으로 대체될 예정)

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run` 으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab` 을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber` 가 선택한 DB UID/GID와 일치한다.
- target host keytab이 `root:root 0400` 이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 해당 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf` 만 전달된다.

구현은 `create_container.py`와 `Kerberos command builder`에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \
  /etc/decs-krb/keytabs/<username>.keytab \
  /etc/decs-krb/refresh.d/<username>.env

sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400
refresh env: root:root 600
ccache dir:  <uid>:<gid> 700
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager

sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \
  klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하는 것이 현재 구현이다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |
  grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)='

docker exec --user <username> <container> \
  sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름

- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4
defaults,vers=4.0,rsize=1048576,wsize=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,
_netdev,exec,nouser 0 0
```

핵심은 source가 FQDN이라는 점이다.

```
올바름: nas.farm.decs.internal:/volume1/share
잘못됨: 100.100.100.120:/volume1/share
잘못됨: 192.168.2.30:/volume1/share
정상: mount option 또는 findmnt 결과의 addr=100.100.100.120
```

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal과 이름이 맞지 않는다. `192.168.2.30`은 NAS 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
findmnt -T /home/tako8/share -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n '/\/home\/tako8\/share/,+4p'
getent hosts nas.farm.decs.internal
```

desired fstab 변경은 [server-state playbook](#)이 소유한다. 이미 mount된 legacy superblock은 유지보수 창에 clean unmount/remount한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd`가 필요하다.

```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')
sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "${short}@FARM.DECS.INTERNAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd`를 `-n`으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n`은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. NAS service account와 KVNO 운영

현재 정책

- NFS SPN 소유자: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```

cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'

```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재

- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 FARM config에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair`는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd`를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

8. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery

무엇을 확인하는가	대표 상태/지표	코드
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

9. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME`, ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

10. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt`, `klist`, checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

11. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Kerberos/NFS 디버깅 로그

이 문서는 Kerberos/NFS에서 발생한 장애와 재현 실험을 축적하는 인덱스다. 정상 상태를 만드는 절차는 [운영 문서](#)에 두고, 여기에는 장애가 어떻게 관측됐고 어떤 가설을 어떤 조건에서 검증했는지 기록한다.

기록 목록

검증일	상태	문제	핵심 결론
2026-07-16	재현 완료, 과거 장애 원인 후보	NFSv4.1 session slot 고착	구형 storage kernel에서 지연된 idmap decode가 NFSv4.1 slot 하나를 영구 INUSE 상태로 남기는 경로를 재현함

상태는 다음 의미로 사용한다.

- **관측**: 증상과 증거는 있지만 원인 경로를 재현하지 못했다.
- **재현 완료**: 통제된 조건에서 같은 kernel 또는 protocol 경로를 다시 만들었다.
- **원인 후보**: 재현된 경로가 과거 장애를 설명하지만 장애 발생 시점의 직접 trace가 없어 동일 원인이었다고 확정할 수 없다.
- **원인 확정**: 장애가 시작된 시점의 증거와 재현 결과가 같은 경로를 가리킨다.

장애 발생 시 먼저 할 일

NFS **hard** mount 관련 process가 D-state에 들어간 경우에는 새로운 **find**, **du**, **stat**, canary I/O를 반복하지 않는다. 각 명령이 새로운 NFS request와 D-state process를 추가할 수 있다.

먼저 NFS 경로를 읽지 않는 local 상태만 수집한다.

```
date -Ins
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS

ps -e -o state=,pid=,ppid=,etimes=,comm=,wchan:48=,args= \
  | awk '$1 ~ /^D/ {print}'

cat /proc/net/rpc/nfs 2>/dev/null || true
cat /run/decs-nfs-forensics/status.json 2>/dev/null || true
cat /var/lib/decs-nfs-gss/recovery.state 2>/dev/null || true
cat /var/lib/decs-nfs-gss/canary.state 2>/dev/null || true

systemctl --no-pager --full status rpc-gssd.service
journalctl -u rpc-gssd.service -b --no-pager -n 200
```

forensics가 배포된 host에서는 자동 snapshot이 생성됐는지 먼저 확인한다. 자동 trigger가 없었다면 local 상태만 읽는 수동 snapshot을 한 번 실행할 수 있다.

```
sudo systemctl start decs-nfs-forensics-snapshot.service
sudo cat /var/lib/decs-nfs-forensics/last_snapshot.state
```

이미 mount된 경로에 D-state caller가 있으면 `rpc-gssd` restart, forced unmount, 반복 remount를 먼저 실행하지 않는다. 증거를 보존한 뒤 storage-network 상태와 kernel wait path를 기준으로 복구 범위를 결정한다.

새 디버깅 문서 작성 형식

새 문제는 한 파일에 다음 내용을 남긴다.

1. **상태와 영향 범위**: 발생 시각, host, kernel, mount source/target/security
2. **증상**: 사용자 증상, process state, wait channel, alert
3. **확인한 사실과 가설**: 관측 사실과 아직 증명하지 못한 추론을 분리
4. **실험 목적**: 어떤 가설을 반증하거나 확인하려 했는지
5. **재현 조건**: server/client, version, cache, 동시성, fault와 안전장치
6. **실행 명령**: 사전 확인, fault 주입, workload, 관측, cleanup 순서
7. **결과와 판정 기준**: 성공/실패를 판단한 trace와 metric
8. **복구와 예방**: 임시 containment, 영구 수정, monitoring 변경
9. **원본 증거**: 결과 README, curated evidence, script와 checksum 링크

운영 환경에서 fault를 주입한 명령은 일반 runbook처럼 제시하지 않고 반드시 실행 일자와 승인·안전 조건을 붙인 **과거 실행 기록**으로 표시한다. 반복 실험은 가능하면 격리 VM harness로 옮긴다.

NFSv4.1 session slot 고착

상태: 2026-07-16에 slot leak 경로 재현 완료. 2026-06-29 LAB5 장애의 강한 원인 후보이지만, LAB5 장애 시작 시점의 storage trace가 없어 과거 장애의 최초 원인으로 확정하지는 않는다.

1. 요약

LAB storage의 구형 kernel `3.10.0-862.el7.x86_64`에서 NFSv4 `SETATTR`가 새로운 group 이름을 UID/GID로 해석하던 중 `rpc.idmapd` 응답이 지연되면 request가 defer될 수 있다. 이때 NFSv4.1 `SEQUENCE`는 slot을 `INUSE`로 표시하지만 response encode와 `nfsd4_sequence_done()`이 실행되지 않아 slot이 반환되지 않는 경로를 실제로 재현했다.

```
sequenceDiagram
    participant C as LAB8 NFS client
    participant N as storage nfsd
    participant I as storage rpc.idmapd

    C->>N: 새 group으로 SETATTR를 요청함
    N->>I: group name의 GID를 조회함
    I-->N: 응답이 8초 동안 지연됨
    N->>N: request를 defer하고 slot을 INUSE로 남김
    I-->>N: 재개 후 GID를 반환함
    C->>N: 같은 SEQUENCE로 다시 요청함
    N-->>C: 이미 INUSE인 slot이라 완료하지 못함
    C->>C: chown이 rpc_wait_bit_killable에서 D-state로 남음
```

한 slot만 leak한 최소 재현에서는 `ForeChannel` waiter가 0일 수도 있다. 따라서 waiter가 없다는 사실만으로 slot 문제를 배제하면 안 된다. session reset이나 recovery가 모든 slot의 drain을 기다리는 상황에서는 고착된 한 slot이 전체 복구를 막을 수 있다.

2. 어떤 증상이었나

2026-07-16 재현에서는 LAB8의 `chown` worker가 173초 이상 D-state에 남았다.

```
chown
-> nfs4_proc_setattr
-> nfs4_call_sync_sequence
-> rpc_wait_bit_killable
```

storage trace에서는 첫 처리와 재방문이 다음처럼 달랐다.

단계	관측
첫 idmap decode	<code>RQ_USEDEFERRAL</code> , <code>RQ_DROPME</code> 가 설정됨
첫 SEQUENCE 확인	<code>seqid=0x5f</code> , <code>slot_seqid=0x5e</code> , <code>INUSE=0</code>
첫 request 종료	dispatcher가 0을 반환했고 encode와 <code>sequence_done</code> 이 실행되지 않음
<code>rpc.idmapd</code> 재개 후	같은 request가 <code>seqid=0x5f</code> , <code>slot_seqid=0x5f</code> , <code>INUSE=1</code> 로 재방문
이후 retry	약 15초마다 같은 <code>INUSE=1</code> 을 확인하고 완료하지 못함

3. 왜 이 실험을 했나

조사는 다음 순서로 가설을 좁혔다.

1. 2026-06-29 LAB5에서는 NFS user home을 읽는 `sshd: ... [priv]` 가 D-state에 누적됐다.
2. 2026-07-10 LAB8에서 TCP/2049 transport를 일시 차단하자 같은 `rpc_wait_bit_killable` 계열 증상을 재현했지만, network가 복구된 뒤 process도 자연 복구됐다.
3. keytab, `rpc-gssd`, readiness와 mount 순서를 바로잡은 clean boot에서도 구형 storage kernel의 session state가 영구 고착될 수 있는지는 별도 검증이 필요했다.
4. upstream의 idmap deferral 수정 내용이 “NFSv4 decode 중 request drop으로 `NFSD4_SLOT_INUSE` 가 해제되지 않는다”는 경로를 설명하므로, 실제 LAB kernel과 mount에서 그 primitive를 최소 workload로 확인했다.

실험 목적은 많은 client를 동시에 멈추게 하는 것이 아니라, **idmap miss 하나가 slot 하나를 반환 불가능하게 만들 수 있는지**를 확인하는 것이었다.

4. 실험 결과

4.1 실제 LAB 환경의 최소 재현

첫 pass에서 `RQ_DROPME` 가 설정된 뒤 slot이 `INUSE` 로 바뀌었지만 encode와 `nfsd4_sequence_done()` 이 호출되지 않았다. 8초 뒤 `rpc.idmapd` 가 재개된 후에도 같은 sequence는 이미 사용 중인 slot으로 판단됐다. client retry도 동일 상태를 반복했고 worker는 D-state에서 끝나지 않았다.

따라서 다음 좁은 결론은 확정할 수 있다.

- 당시 storage kernel에서는 idmap decode deferral이 NFSv4.1 session slot 하나를 leak할 수 있다.
- keytab 발급, `rpc-gssd` 시작 순서나 client mount 순서만 고쳐서는 이 server-side kernel 경로를 제거할 수 없다.
- 한 slot leak은 재현했지만 LAB5에서 나중에 관측한 전체 session recovery/drain stall까지 이 최소 실험 하나로 재현한 것은 아니다.
- LAB5 장애 시작 시점의 trace가 없으므로 어떤 실제 operation이 최초 idmap miss를 만들었는지는 알 수 없다.

4.2 A-G: 8초 idmap 지연 kernel matrix

최소 재현과 같은 `idmap-setattr`, NFSv4.1, `sec=krb5p` 조건에서 server와 client kernel 조합을 바꿔 각 5회 실행했다. client OS는 모두 Ubuntu 22.04.5 LTS이며, F/G만 client kernel을 `6.18.38` 로 올렸다.

Case	NFS server OS	Server kernel	Client kernel	결과
A	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
B	CentOS Linux 7.9.2009	<code>3.10.0-1160.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
C	CentOS Stream 10	<code>6.12.74</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
D	CentOS Stream 10	<code>6.12.75</code>	<code>6.8.0-134-generic</code>	자동 복구 5/5

Case	NFS server OS	Server kernel	Client kernel	결과
E	CentOS Stream 10	6.12.0-248.el10.x86_64	6.8.0-134-generic	자동 복구 5/5
F	CentOS Linux 7.9.2009	3.10.0-862.el7.x86_64	6.18.38	slot 고착 5/5
G	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.18.38	slot 고착 5/5

구형 server kernel에서는 client kernel을 6.18.38 로 올린 F/G도 모두 고착됐다. 반대로 idmap/slot fix가 포함된 D와 signed Stream kernel E는 모두 자동 복구했다. 이 결과는 문제와 복구 여부를 가르는 주된 변수가 client가 아니라 **NFS server kernel의 idmap deferral 처리**임을 지지한다.

4.3 D/E: 120·300·600초 장시간 idmap 지연 추가 실험

8초 지연만으로는 수정된 kernel이 짧은 지연에서만 우연히 복구한 것인지 판단하기 어려웠다. 그래서 자동 복구의 D/E만 대상으로 idmap cache miss 응답을 120초, 300초, 600초까지 지연하고 각 조건을 3회씩 다시 실행했다.

Case	Server kernel	idmap 지연	원래 chgrp	신규 RPC	최종 snapshot D / FC / lease
D-120s	6.12.75-nfs-slot	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0*
D-300s	6.12.75-nfs-slot	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
D-600s	6.12.75-nfs-slot	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0
E-120s	6.12.0-248.el10.x86_64	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-300s	6.12.0-248.el10.x86_64	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-600s	6.12.0-248.el10.x86_64	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0

여기서 **FC**는 NFSv4.1 ForeChannel waiter 수다. 600초 조건의 **EINVAL**은 원래 **chgrp** 요청의 application 결과이며, slot 고착 판정과 분리해야 한다. 같은 run에서 idmap 지연을 해제한 뒤 실행한 신규 RPC는 모두 성공했고, 최종 snapshot의 D-state, ForeChannel waiter와 **lease_expired**도 모두 0이었다. 따라서 600초에서 원래 작업이 실패했더라도 **session slot은 반환됐고 후속 요청을 막지 않았다**.

* D-120s 2회차의 2초 sampler는 종료 직전 D-state 1을 한 번 기록했다. 그러나 직후 final snapshot은 D-state 0이었고, 신규 RPC 성공, ForeChannel 0, **lease_expired=0**, analyzer **verdict=pass**였다. 지속된 NFS slot 고착으로 판정하지 않았다.

5. 실제 재현 조건

항목	2026-07-16 조건
client	LAB8, 100.100.100.108
server	lab-storage.lab.decs.internal, 100.100.100.100

항목	2026-07-16 조건
mount	/home/tako8/share , NFSv4.1, sec=krb5p , hard
storage kernel	3.10.0-862.el7.x86_64
client 시작 상태	D-state 0, NFS RPC task 0, readiness ready
test identity	LAB8 local group decs_idmap_repro_424242 , GID 424242
test object	/home/tako8/share/.decs-idmap-slot-repro-20260716
fault	storage rpc.idmapd 에 8초간 SIGSTOP
workload	test file에 GID 424242 를 지정하는 chown(2) 한 번
안전장치	storage-local 8초·20초 SIGCONT timer를 먼저 예약
금지한 복구	재현 후 client reboot, unmount, signal, service restart를 하지 않음

boot 순서도 별도로 확인했다. LAB8은 rpc-gssd active → Kerberos/NFS readiness 완료 → mount 순서였고, 기존 /etc/krb5.keytab 의 KVNO는 2였다. 따라서 이번 결과를 keytab 생성 누락이나 잘못된 mount 시작 순서로 설명할 수 없다.

6. 당시 실행한 명령

경고: 아래 명령은 2026-07-16의 승인된 실제 실행 기록이다. storage의 rpc.idmapd 를 멈추므로 일반 점검이나 운영 재현 절차로 다시 실행하면 안 된다. 재실험은 8절의 격리 VM harness를 사용한다. 특히 kprobe 인자 register는 당시 x86_64 kernel symbol에만 맞춘 값이라 다른 kernel에서 그대로 사용하지 않는다.

6.1 client 사전 확인과 test 값 준비

LAB8에서 D-state와 RPC task가 없는지 확인한 뒤, storage에 없던 group 이름을 client가 encode하도록 local group과 test file을 준비했다.

```

ps -eo stat= | awk '$1 ~ /^D/ {n++} END {print "d_state=" (n+0)}'
grep -c '^RPC task' /proc/net/rpc/nfs 2>/dev/null || true
findmnt -T /home/tako8/share -n -o TARGET,SOURCE,FSTYPE,OPTIONS

sudo groupadd -g 424242 decs_idmap_repro_424242

sudo bash -c '
p=/home/tako8/share/.decs-idmap-slot-repro-20260716
rm -f "$p"
touch "$p"
chmod 0666 "$p"
stat -c "path=%n uid=%u gid=%g mode=%a ino=%i" "$p"
'

```

fault 전에 GID 424241 을 사용한 정상 `chown` 을 실행해 같은 경로가 idmap 지연 없이는 encode와 `sequence_done` 까지 완료되는 것을 먼저 확인했다.

```

sudo python3 -c 'import os; p="/home/tako8/share/.decs-idmap-slot-repro-20260716"; print("before", os.stat(p).st_gid); os.chown(p, -1, 424241); print("after", os.stat(p).st_gid)'

```

6.2 storage trace 설정

당시 storage에서 idmap decode, slot 검사, dispatcher return, response encode와 slot 반환 여부를 한 trace instance에 기록했다.


```

T=/sys/kernel/debug/tracing
I="$T/instances/decs-idmap-slot-repro"
sudo mkdir -p "$I"

for event in uid gid check_slot dispatch dispatch_ret encode sequence sequence_done; do
    echo "-:decs_idmap/$event" | sudo tee -a "$T/kprobe_events" >/dev/null || true
done

echo 'p:decs_idmap/uid nfsd_map_name_to_uid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/gid nfsd_map_name_to_gid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/check_slot check_slot_seqid seqid=%di:u32 slot_seqid=%si:u32 slot_inuse=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/dispatch nfsd_dispatch rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'r:decs_idmap/dispatch_ret nfsd_dispatch ret=$retval:s32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/encode nfs4svc_encode_compoundres rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence nfsd4_sequence rqstp=%di cstate=%si seq=%dx' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence_done nfsd4_sequence_done resp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null

echo 0 | sudo tee "$I/tracing_on" >/dev/null
echo 0 | sudo tee "$I/events/enable" >/dev/null
sudo sh -c ": > '$I/trace'"
echo 8192 | sudo tee "$I/buffer_size_kb" >/dev/null

for event in \
    decs_idmap/uid decs_idmap/gid decs_idmap/check_slot decs_idmap/dispatch \
    decs_idmap/dispatch_ret decs_idmap/encode decs_idmap/sequence \
    decs_idmap/sequence_done sunrpc/svc_process sunrpc/svc_defer \
    sunrpc/svc_revisit_deferred sunrpc/svc_drop sunrpc/svc_send
do
    echo 1 | sudo tee "$I/events/$event/enable" >/dev/null
done
echo 1 | sudo tee "$I/tracing_on" >/dev/null

```

6.3 fault와 단일 workload 실행

storage에서 반드시 자동 재개 timer 두 개가 active인지 확인한 뒤 `rpc.idmapd`를 멈췄다.

```
pid="$(pgrep -xo rpc.idmapd)"

sudo systemd-run --unit=decs-idmapd-auto-resume-8s \
  --on-active=8s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"
sudo systemd-run --unit=decs-idmapd-auto-resume-20s \
  --on-active=20s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"

sudo systemctl is-active --quiet decs-idmapd-auto-resume-8s.timer
sudo systemctl is-active --quiet decs-idmapd-auto-resume-20s.timer
sudo kill -STOP "$pid"
```

그 직후 LAB8의 별도 systemd worker에서 `chown(2)` 한 번만 실행했다.

```
sudo systemd-run --unit=decs-idmap-slot-repro-worker --no-block \
  /usr/bin/python3 -c \
  "import os; os.chown('/home/tako8/share/.decs-idmap-slot-repro-20260716', -1, 424242)"
```

6.4 결과 확인과 trace 보존

새로운 NFS I/O를 만들지 않고 worker, local kernel state와 이미 동작 중이던 forensics state를 확인했다.

```
systemctl --no-pager --full status decs-idmap-slot-repro-worker.service
ps -eo pid=,ppid=,stat=,comm=,wchan:32=,args= \
  | awk '$3 ~ /^D/ {print}'
grep -E 'lease_expired|RPC task|xprt_pending|ForeChannel' \
  /proc/net/rpc/nfs 2>/dev/null || true

cat /run/decs-nfs-forensics/status.json
cat /var/lib/decs-nfs-gss/recovery.state
cat /var/lib/decs-nfs-gss/canary.state
```

storage trace는 다음 pattern을 기준으로 추렸다.

```
I=/sys/kernel/debug/tracing/instances/decs-idmap-slot-repro
sudo sh -c "echo 0 > '$I/tracing_on'"
sudo cp "$I/trace" /var/tmp/decs-idmap-slot-repro-20260716.trace

sudo grep -E \
  'decs_idmap_gid|svc_defer|svc_revisit_deferred|svc_drop:|check_slot|sequence_done|dispatch_ret' \
  /var/tmp/decs-idmap-slot-repro-20260716.trace
```

7. Monitoring과 운영 대응

재현 당시 guardrail은 의도대로 동작했다.

- forensics watcher가 `nfs_d_state` 이유로 incident snapshot을 보존했다.
- recovery는 `status=blocked`, `diagnosis=nfs_d_state_detected`, `retry_inhibited=1`로 바뀌었다.
- canary도 같은 진단으로 차단돼 추가 NFS request를 만들지 않았다.
- packet ring과 kernel trace는 계속 증거를 수집했다.

운영에서 같은 증상이 보이면 다음 순서를 따른다.

1. NFS path를 읽는 `find`, `du`, `stat`, canary를 추가 실행하지 않는다.
2. D-state stack, `/proc/net/rpc/nfs`, forensics status와 snapshot을 보존한다.
3. mount가 이미 있으면 자동 remount와 `rpc-gssd` 반복 restart를 차단한다.
4. storage kernel, `rpc.idmapd`, network와 NFS session 증거를 함께 확인한다.
5. 상태가 자연 복구되지 않으면 유지보수 창외 client reboot 또는 storage 조치를 수동 결정한다. reboot는 상태를 지울 뿐 원인 수정은 아니다.
6. 영구 조치는 해당 idmap deferral fix가 포함된 storage kernel로 올리고 격리 회귀 실험을 통과시키는 것이다.

운영 mount를 `soft`로 바꾸는 방식은 timeout을 application error로 노출하고 data integrity 위험을 만들 수 있어 해결책으로 사용하지 않는다.

8. 격리 VM에서 다시 검증하는 방법

반복 실험은 LAB8 local disk의 `decs-nfs-slot-isolated` network에서 수행한다. 이 network는 forwarding이 없고 `100.100.100.0/24`, `192.168.1.0/24`와 운영 storage hostname을 거부한다. VM disk, trace와 result도 NFS share가 아니라 LAB8 local filesystem에 둔다.

kernel 비교 기준은 다음과 같다.

Case	Server kernel	Client kernel	목적
A	CentOS 7 3.10.0-862	Ubuntu 6.8.0-134	운영 취약 baseline
B	CentOS 7 3.10.0-1160	Ubuntu 6.8.0-134	CentOS 7 최종 kernel 비교

Case	Server kernel	Client kernel	목적
C	upstream 6.12.74	Ubuntu 6.8.0-134	fix 이전 positive control
D	upstream 6.12.75	Ubuntu 6.8.0-134	idmap/slot fix 비교 control
E	signed Stream 6.12.0-248.el10	Ubuntu 6.8.0-134	운영 후보
F	CentOS 7 3.10.0-862	kernel.org 6.18.38	새 client kernel 비교
G	CentOS 7 3.10.0-1160	kernel.org 6.18.38	새 client kernel 비교

준비와 격리는 VM lab README의 절차를 따른다.

```
cd /home/jy/server_manage/kerberos-nfs/test/lab-kerberos-poc/vm-slot-regression

sudo ./bin/provision_lab8_vms.sh "$PWD"
./bin/configure_vm_stack.sh
sudo ./bin/prefetch_kernel_matrix.sh
sudo ./bin/verify_kernel_sources.sh
sudo ./bin/build_upstream_kernels_vm.sh
sudo ./bin/prepare_centos7_domains.sh "$PWD"
sudo ./bin/isolate_test_vms.sh
```

단일 비교는 같은 idmap-setattr, sec=krb5p, 8초 조건을 case별로 실행한다.

```
./bin/switch_kernel_case.sh A
NFS_SLOT_EXPECT_VULNERABLE=1 \
./bin/run_single_case.sh A krb5p idmap-setattr 8 1

./bin/switch_kernel_case.sh D
NFS_SLOT_EXPECT_VULNERABLE=0 \
./bin/run_single_case.sh D krb5p idmap-setattr 8 1
```

먼저 실행 수만 확인하려면 plan-only mode를 쓴다.

```
MATRIX_PLAN_ONLY=1 \
MATRIX_CASES='A C D E' \
MATRIX_SECURITY='krb5p' \
MATRIX_SCENARIOS='idmap-setattr' \
MATRIX_DURATIONS='8' \
MATRIX_REPETITIONS=3 \
./bin/run_matrix.sh
```

취약 case가 고착되면 harness는 evidence를 archive한 뒤 **격리 VM만** reset한다. 그 reset은 자동 복구 성공으로 계산하지 않는다. 운영 LAB8 host와 운영 NFS mount는 이 harness의 복구 대상이 아니다.

9. 원본 증거와 코드

- 2026-07-16 실제 재현 결과
- curated trace evidence
- 2026-07-10 NFS transport fault 실험
- 격리 VM slot regression lab
- single-case harness
- result analyzer
- upstream stable patch 설명
- NFS forensics 구현

보존된 storage raw trace의 SHA-256은 `5fe7fbab2b8aa712ff71afe5156438b4e880edcc39cf16a00592e22d578bac59` 다.